Assignment 01: Input Processing, Decision Structures and Boolean Logic

CSci 233: Application Program Development w/ Python

Objectives

- Learn how to input and process data from a user in Python.
- Learn about declaring and using simple variables and data types in an application program.
- Practice using the print function to display output results from programs.
- Practice performing calculations using standard math operations to perform calculations in an application.
- Learn how to use string formatting using formatted strings and formatting values in Python
- Practice using conditional statements to perform calculations.
- Practice and learn about more advanced conditional statements like nested decision structures.
- Understand how to use logical operators like `and` `or` and `not` to build boolean expressions for calculations.

Description

This assignment asks you complete from 6 to 10 simple Python scripts using the concepts learned from chapter 2 and 3 of our class textbook. These chapters cover the basics of performing Input / Output using Python, using variables and performing calculations, and creating conditional statements (`if` / `elif` / `else`) to make decisions in programs.

Assignment Prerequisites and Setup

You will need to have completed the following setup before you can begin this assignment.

Before performing any assignment in this class, you need to have the following tasks already completed.

1. You need to have `git` tools installed on your system so that you can successfully clone repositories and create commits.
2. You need to have a GitHub account created.
   - You need to have successfully created a ssh key so that you can authenticate with and push commits back to git.
3. You need a working Python 3 distribution installed on the system you will work on these assignments with, that you can run Python scripts and the Python IDLE interface within.

See our class `Getting Started` for links on setting up these needed tools and configurations if you have not yet completed them before beginning this assignment.

Assignment Questions

Start by cloning the assignment repository that was created for you when you accepted the assignment. There are 8 mostly empty Python script files in the top level of this assignment named `q01.py` through `q08.py`.

You should answer each of the following questions by writing appropriate Python code to solve the stated task. Once you are satisfied with your answer, you should create a commit and push it back to your GitHub repository for grading. In general you should do each question as a separate commit (see Git Best Practices).

Each commit you push to GitHub will create a release that will be graded. Look at the `Releases` on the right hand side of your `Code` page to see a summary of your grade results. Also the instructor will give feedback for all assignments in your `Feedback` Pull request. See the `Feedback` pull request listed on GitHub on the `Pull requests` tab.

You can and should test your code locally before creating commits and submitting it to GitHub. You can do this for each question by opening a terminal and changing into your project directory, and running the following to perform the tests for a question:

```
python3 test/test-question.py q01.py
```

This will run all of the tests for you for question 01, and will tell you if you are successfully passing, or if you are failing one or more of the tests.

All tests in these assignments will use a simple comparison of your programs output to the expected correct output. This means that you need to exactly match the expected output of the tests to receive full credit for a question. Changes in the amount of whitespace are usually ignored, except when specific formatting is required. However things like misspelling of words, or added punctuation, or using capital letters when not asked for, etc. will all cause tests to fail. And of course if you perform a calculation incorrectly or do not perform some of the parts of the question, these will also cause your tests to fail. You can look at the expected output for all tests in the `test/data` subdirectory files.

Question 1 Total Purchase (3 points)

Write a program that will ask the user to enter the amount of a purchase. The program should then compute the state and county sales tax. Assume the state sales tax is 5 percent and the county sales tax is 2.5 percent. The program should display the amount of the purchase, the state sales tax, the county sales tax, the total sales tax, and the total of the sale (which is the sum of the amount of purchase plus the total sales tax).

Requirement: You should use Python formatted strings for this question, and to perform all of the output formatting in the questions in this assignment. Also all dollar amounts are required to be displayed rounded to two decimal places (e.g. dollars and cents).

Hint: A format specifier of `{variable:0.2f}` will format a floating point variable and round and display it using 2 decimal digits.

Hint: Use the value 0.025 to represent 2.5 percent and 0.05 to represent 5 percent.

The following discussion is specific to how to correctly complete question 1 to pass your autograder tests, but applies to all of the following questions in this assignment.

For example lets say you have written your `q01.py` and want to see if it works. Open a terminal or run your script however you normally do it. For example, lets say when you run it you get the following:

```
$ python3 q01.py
What is the amount of the purchase? 40.00
The amount of the purchase 40.00
State sales taxes 2.00
County sales taxes 1.00
Total sales taxes 3.00
Total sales amount 43.00
```

Here when prompted to enter the amount, the user entered 40.00. This is the purchase amount used in the first test for question 01. You can run the given tests for this assignment like this:

```
$ python3 test/test-question.py q01.py 1
Question <q01> Test 01:  FAILED
003 student:  <State sales taxes 2.00>
003 expected: <State sales tax 2.00>
004 student:  <County sales taxes 1.00>
004 expected: <County sales tax 1.00>
005 student:  <Total sales taxes 3.00>
005 expected: <Total sales tax 3.00>
```

You will see that we asked to run test 01 for the q01.py script. The test failed. The output is saying that, for example, on line 3 the student output the first line, but the expected output was as shown on the second line (e.g. the student output taxes instead of tax). You can see the expected output for this test in the `test/data/q01-01-expected.out` file. When you perform the exercises in this class, you will need to match the expected output or test results exactly

in order to pass the tests for a question, so you may need to look into the expected output for a question and format your output accordingly.

If we fix our program to output this instead:

```
$ python3 q01.py
What is the amount of the purchase? 40.00
The amount of the purchase 40.00
State sales tax 2.00
County sales tax 1.00
Total sales tax 3.00
Total sales amount 43.00
```

and now run our test 01 for question 01 again, you should see:

```
$ python3 test/test-question.py q01.py 1
Question <q01> Test 01:  PASSED

 ======================================================================
 All attempted question tests passed  (001 passed of 001 tests attempted)
```

There will usually be several tests for each question. You can run all tests by simply not specifying a specific test number to run, for example, if your solution is correct for question 01 you should get

```
$ python3 test/test-question.py q01.py
Question <q01> Test 01:  PASSED
Question <q01> Test 02:  PASSED
Question <q01> Test 03:  PASSED

 ======================================================================
 All attempted question tests passed  (003 passed of 003 tests attempted)
```

Once you have your code working for a question, you are still not done. You need to submit your code for grading. At this point you should create a commit of your working `q01.py` code, and push it to your GitHub classroom assignment repository. Once a commit is successfully pushed, you should be able to view its results in your `Feedback` pull request, and get a detailed report by finding the `Release` for this commit and viewing the results of the autograder there.

Question 2 Miles-per-Gallon (3 points)

A car's miles-per-gallon (MPG) can be calculated with the following formula:

$$MPG = \text{Miles driven} \div \text{Gallons of gas used}$$

Write a program that asks the user for the number of miles driven and the gallons of gas used. It should calculate the car's MPG and display the result.

We will not repeat the discussion again about how to get the correct output. The calculated miles per gallon in this problem needs to be formatted to have a single digit displayed after the decimal point. Once you have question 02, make sure you make a commit and push it to your GitHub repository for autograding. Remember it is better to make separate commits of each question when you finish it, and it may be a requirement in future assignments to perform separate commits for questions or tasks.

Question 3 Male and Female Percentages (3 points)

Write a program that asks the user for the number of males and the number of females registered in a class. The program should display the percentage of males and females in the class.

Hint: Suppose there are 8 males and 12 females in a class. There are 20 students in the class. The percentage of males can be calculated as $8 \div 20 = 0.4$, or 40.0%. The percentage of females can be calculated as $12 \div 20 = 0.6$, or 60.0%.

Once you are finished test your program locally and then create and push a commit to have it autograded.

Question 4 Compound Interest (5 points)

When a bank account pays compound interest, it pays interest not only on the principal amount that was deposited into the account, but also on the interest that has accumulated over time. Suppose you want to deposit some money into a savings account, and let the account earn compound interest for a certain number of years. The formula for calculating the balance of the account after a specified number of years is:

$$A = P(1 + \frac{r}{n})^{nt}$$

The terms in the formula are: - A is the amount of money in the account after the specified number of years. - P is the principal amount that was originally deposited into the account. - r is the annual interest rate. - n is the number of times per year that the interest is compounded. - t is the specified number of years.

Write a program that makes the calculation for you. The program should ask the user to input the following:

- The number of times per year that the interest is compounded (For example, if interest is compounded monthly, enter 12. If interest is compounded quarterly, enter 4.)
- The number of years the account will be left to earn interest

Once the input data has been entered, the program should calculate and display the amount of money that will be in the account after the specified number of years.

Note: The user is expected to enter the interest rate as a percentage. For example, 2 percent is entered as 2 when prompted, not as .02. Your program will then have to divide the input by 100 to move the decimal point to the correct position.

Note You are again working with dollar amounts here, so round all dollar amounts to two decimal places when displayed.

Question 5 February Days (7 points)

The month of February has typically 28 days. But if it is a leap year, February has 29 days. Write a program that asks the user to enter a year. The program should then display the number of days in February that year. Use the following criteria to identify leap years:

Determine whether the year is divisible by 100. If it is, then it is a leap year if and only if it is also divisible by 400. For example, 2000 is a leap year, but 2100 is not.

If the year is not divisible by 100, then it is a leap year if and only if it is divisible by 4. For example, 2008 is a leap year, but 2009 is not.

Question 6 Mass and Weight (3 points)

Scientists measure an object's mass in kilograms (kg) and its weight in newtons. If you know the amount of mass of an object in kilograms, you can calculate its weight in newtons with the following formula:

$$\text{weight} = \text{mass} \times 9.8$$

Write a program that asks the user to enter an object's mass, then calculates its weight. If the object weighs more than 500 newtons, display a message indicating that it is too heavy. If the object weighs less than 100 newtons, display a message indicating that it is too light. If the weight is in between, display a message indicating that the weight is just right.

Question 7 Roulette Wheel Colors (3 points)

On a roulette wheel, the pockets are numbered from 0 to 36. The colors of the pockets are as follows:

- Pocket 0 is green.

- For pockets 1 through 10, the odd-numbered pockets are red and the even-numbered pockets are black.
- For pockets 11 through 18, the odd-numbered pockets are black and the even-numbered pockets are red.
- For pockets 19 through 28, the odd-numbered pockets are red and the even-numbered pockets are black.
- For pockets 29 through 36, the odd-numbered pockets are black and the even-numbered pockets are red.

Write a program that asks the user to enter a pocket number and displays whether the pocket is green, red, or black. The program should display an error message if the user enters a number that is outside the range of 0 through 36.

Hint: You can use the modulus operator `%` to test and help determine if a pocket is odd-numbered or even-numbered.

Question 8 Areas of Rectangles (3 points)

The area of a rectangle is the rectangle's length times its width. Write a program that asks for the length and width of two rectangles. The program should tell the user which rectangle has the greater area, or if the areas are the same.

Assignment Conclusion / Checklist

Hopefully you have created commits after completing each question and pushed them successfully to your assignment repository. Make sure that you do the following for this and all class assignments.

- Check your autograder results after pushing your commits. Look in your `Feedback` pull request, and in the detailed autograder report generated for your release(s) made for each commit.
- You should never close or merge your `Feedback` pull request, as is mentioned in the comment. The instructor will evaluate your assignments, and may give you feedback about your code or work. Check here after the assignment is returned for a code review and assignment comments.
- In general this class will ask you fo follow and use good Python style as specified by the official Python PEP 8 Style guide. You should at least pay attention to
  - All indentation is required to be 4 spaces with no tabs in files.
  - Maximum line lengths should be usually observed, do not generally extend code past the 79 character column in a file.
  - Follow the suggestions for breaking long lines in code (e.g. line break before binary operators in a long expressions).
  - Prefer single quotes for all string in code for class assignments, unless you need a string with a single quote, for example to add an apostrophe 's.
  - Follow Pep8 stile for whitespace in expressions and statements.
    For example, usually a single space should go before and after all binary operators in expressions.
  - We encourage the use of function annotations in Python code.
- Also in general in this class we will ask you to follow Git Commit Best Practices
- Strive to use Meaningful Names for variables, constants and functions in your code. You are required to follow Pep8 naming guidelines, which means `snake_case` names for variables and functions and `SCREAMING_SNAKE_CASE` for constants. Python style switches to `PascalCase` for class names.

Make sure you are checking review comments and code review given for your class assignments. We may start by making suggestions where your style or practices could be improved. As the class progresses, some of these suggestions may become requirements, especially for students who are repeatedly making the same style or practice error and are not following feedback to correct a noted issue in future assignments. Assignments may be left ungraded in some cases for style or practice issues until corrected, and/or may have points removed or receive a 0 grade if issues continue after receiving multiple feedback that are repeatedly ignored and not corrected.

Additional Information

The following are suggested online materials you may use to help you understand the tools and topics we have introduced in this assignment.

- Python PEP 8 Style guide
- Git Commit Best Practices
- Git Best Practices
- Best Practices to Write Readable and Maintainable Code: Choosing meaningful names
- How to Write Better Names for your Variables, Functions and Classes